Forum Donate

Learn to code — free 3,000-hour curriculum

MAY 3, 2023 / #C PROGRAMMING

How to Use Pointers in C Programming

Valentine Gatwiri

If you are learning C programming, you have probably heard the term "pointer" before.

Pointers are one of the most important and powerful features of the C programming language. They allow us to manipulate memory directly, which can be very useful in many programming scenarios.

In C, a pointer is simply a variable that holds a memory address. We can think of it as a way to refer to a specific location in memory.

How to Declare a Pointer

To declare a pointer variable in C, we use the asterisk `*` symbol before the variable name. There are two ways to declare pointer variables in C:

Forum Donate

Learn to code — free 3,000-hour curriculum

```
int* p;
```

Both of these declarations are equivalent and they declare a pointer variable named "p" that can hold the memory address of an integer.

However, it's important to note that if you declare multiple variables in a single statement, you need to include the asterisk before each variable name to indicate that they are all pointers. For example:

```
int *p, *q, *r;
```

This declares three pointer variables named "p", "q", and "r" that can hold the memory address of an integer.

How to Initialize a Pointer

When we declare a pointer variable, it does not automatically point to any particular memory location. To initialize a pointer to point to a specific variable or memory location, we use the ampersand `&` operator to get the address of that variable.

For example, to initialize the pointer `p` to point to an integer variable called `x`, we would write:

Forum Donate

Learn to code — free 3,000-hour curriculum

This sets the value of `p` to be the memory address of `x`.

How to Dereference a Pointer

Once we have a pointer that points to a specific memory location, we can access or modify the value stored at that location by dereferencing the pointer.

To dereference a pointer, we use the asterisk `*` symbol again, but this time in front of the pointer variable itself. For example, to print the value of the integer that `p` points to, we would write:

```
printf("%d\n", *p);
```

What Does "Pointer to a Pointer" Mean?

A pointer can also point to another pointer variable. This is known as a "pointer to a pointer". We declare a pointer to a pointer by using two asterisks `**`. For example:

```
int x = 42;
int *p = &x;
int **q = &p;
```

Forum Donate

Learn to code — free 3,000-hour curriculum

How to Pass Pointers to Functions

We can pass pointers to functions as arguments, which allows the function to modify the value of the original variable passed in. This is known as "passing by reference".

To pass a pointer to a function, we simply declare the function parameter as a pointer. For example:

```
void increment(int *p) {
    (*p)++;
}

int main() {
    int x = 42;
    int *p = &x;
    increment(p);
    printf("%d\n", x); // prints 43
    return 0;
}
```

Here, the `increment` function takes a pointer to an integer (`int *p`) and increments the value of the integer by one (`(*p)++`).

In `main()` , we declare the integer `x` and a pointer `p` that points to `x` . We then call the `increment` function, passing in the `p` pointer. After the function call, `x` has been incremented to `43` .

Forum Donate

Learn to code — free 3,000-hour curriculum

One of the most powerful uses of pointers in C is for dynamic memory allocation. This allows us to allocate memory at runtime, rather than at compile time.

We use the `malloc` function to dynamically allocate memory, and it returns a pointer to the allocated memory. For example:

```
int *p = (int*)malloc(sizeof(int));
```

Here, `p` is a pointer to an integer that has been allocated using `malloc`. The `sizeof` operator is used to determine the size of an integer in bytes.

After allocating memory, we can use the pointer variable like any other pointer. When we are finished with the memory, we should free it using the `free` function. For example:

```
free(p);
```

This frees up the memory that was allocated to `p`.

What is Pointer Casting?

Sometimes you may need to cast a pointer from one type to another. You can do this using the `(type *)` syntax. For example:

Forum Donate

Learn to code — free 3,000-hour curriculum

Here, `p` is cast to a pointer to a `double` type.

How Does Pointer Arithmetic Work?

Because pointers hold memory addresses, we can perform arithmetic operations on them to move them to different memory locations.

For example, we can increment a pointer to move it to the next memory location. This is often used in array operations, where we use a pointer to access elements of an array.

For example, to print the first element of an integer array using a pointer, we could write:

```
int arr[] = {1, 2, 3};
int *p = arr; // p points to the first element of arr
printf("%d\n", *p); // prints 1
```

Here, `p` is set to point to the first element of the `arr` array, and `*p` dereferences the pointer to get the value of the first element (which is `1`).

How to Use Pointer Arrays

We can also declare arrays of pointers in C. For example:

Forum Donate

Learn to code — free 3,000-hour curriculum

This declares an array of three pointers to integers. Each element of the array can point to a separate integer variable.

Pointer Arithmetic and Arrays

We can use pointer arithmetic to access elements of an array. For example:

```
int arr[] = {1, 2, 3};
int *p = arr; // p points to the first element of arr
printf("%d\n", *(p + 1)); // prints 2
```

Here, `p` is set to point to the first element of the `arr` array. We can use pointer arithmetic to access the second element of the array (`*(p + 1)`), which is `2` .

Example of How to Use Pointers

Here's an example program that demonstrates some of the concepts we've discussed:

```
#include <stdio.h>
#include <stdlib.h>

void increment(int *p) {
    (*p)++;
}
```

Forum Donate

Learn to code — free 3,000-hour curriculum

```
    printf("x = %d\n", x); // prints x = 42
    increment(p);
    printf("x = %d\n", x); // prints x = 43

    int *arr = (int *)malloc(3 * sizeof(int));
    arr[0] = 1;
    arr[1] = 2;
    arr[2] = 3;
    int *q = arr;
    printf("arr[0] = %d\n", *q); // prints arr[0] = 1
    q++;
    printf("arr[1] = %d\n", *q); // prints arr[1] = 2
    q++;
    printf("arr[2] = %d\n", *q); // prints arr[2] = 3
    free(arr);

    return 0;
}
```

Output:

This program demonstrates several concepts related to pointers.

Forum Donate

Learn to code — free 3,000-hour curriculum

The function modifies the value of `x` by incrementing it by one. We then printed the value of `x` before and after the function call to demonstrate that `x` has been incremented.

Next, we used dynamic memory allocation to allocate an array of three integers. We set the values of the array elements using pointer arithmetic (`arr[0] = 1`, `arr[1] = 2`, etc.). We then declared a pointer `q` that points to the first element of the array. Furthermore, we used pointer arithmetic to access and print the values of each element of the array.

Finally, we freed the memory that was allocated to the array using the `free` function.

This program demonstrates how pointers can be used to modify the value of a variable, access elements of an array using pointer arithmetic, and dynamically allocate and free memory.

Common Pointer Errors

Pointers can be tricky to work with, and they can lead to some common errors.

One common error is using an uninitialized pointer. If you declare a pointer variable but do not initialize it to point to a valid memory location, you may get a segmentation fault or other error when you try to dereference the pointer.

Another common error is dereferencing a null pointer, which can also cause a segmentation fault.

Forum Donate

Learn to code — free 3,000-hour curriculum

dereference it as a pointer to a character, you may get unexpected results or errors.

Conclusion

Pointers are a powerful tool in C programming, but they can be a bit tricky to work with. With practice and patience, you can master pointers and use them to manipulate memory and work with complex data structures.

Thank you for reading!

Valentine Gatwiri

A curious full-stack web developer. I love solving problems using software development and representing Data in a meaningful way. I like pushing myself and taking up new challenges.

If you read this far, tweet to the author to show them you care.

Tweet a thanks

Learn to code for free. freeCodeCamp's open source curriculum has helped more than 40,000 people get jobs as developers.

Get started

ADVERTISEMENT

Forum Donate

Learn to code — free 3,000-hour curriculum

freeCodeCamp is a donor-supported tax-exempt 501(c)(3) charity organization (United States Federal Tax Identification Number: 82-0779546)

Our mission: to help people learn to code for free. We accomplish this by creating thousands of videos, articles, and interactive coding lessons - all freely available to the public. We also have thousands of freeCodeCamp study groups around the world.

Donations to freeCodeCamp go toward our education initiatives, and help pay for servers, services, and staff.

You can make a tax-deductible donation here.

Trending Guides

JS Substring Tutorial
Bubble Sort Algorithm
HTML Background Image
What is Data Science?
SQL Subquery in SELECT
Rename a File in Linux
Git Remove Last Commit
What is a Java Hashmap?
CRUD Operations Defined
Git Push Local to Remote
UX vs UI
VLOOKUP in Excel
Big O Cheatsheet
Git Rename Branch
Excel Pivot Table
What is Alt Text?
HTML Dropdown Menu
Python Reverse List
Compare Arrays in JS
HTML Background Color

Forum Donate

Learn to code — free 3,000-hour curriculum

CSS Selectors Cheatsheet
Computer Programmer Salary
Sort Dict by Value Python
Dual Boot Windows + Ubuntu
Change Text Color in HTML
What is Information Systems?

Our Charity

About Alumni Network Open Source Shop Support Sponsors Academic Honesty

Code of Conduct Privacy Policy Terms of Service Copyright Policy